THE 8086 MICROPROCESSOR
&
ITS ARCHITECTURE

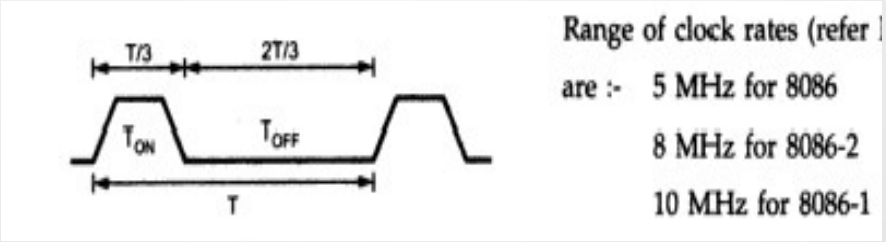
CHAPTER-TWO

Outlines

- Overview of 8086
- Architecture of the 8086
- The Bus Interface Unit (BIU)
- The Execution Unit (EU)
- Register Organization
  - General Purpose Registers
  - Segment Registers
  - Pointers & Index Registers
  - Flag Registers
- Memory Segmentation

The Overview of 8086

- The 8086 is a 16-bit microprocessor
- Has a 16-bit data bus, R/W data 16/8 bit at a time
- Has a 20-bit address bus, so it can access ..byte wide
- Can generate 16-bit I/O address, it can access I/O ports
- Has a multiplexed address and data bus,
  - Reduce the no. of pins required
  - But slow dawn the transfer rate
- Requires one phase clock with a 33% duty cycles (minimum) to provide optimized internal



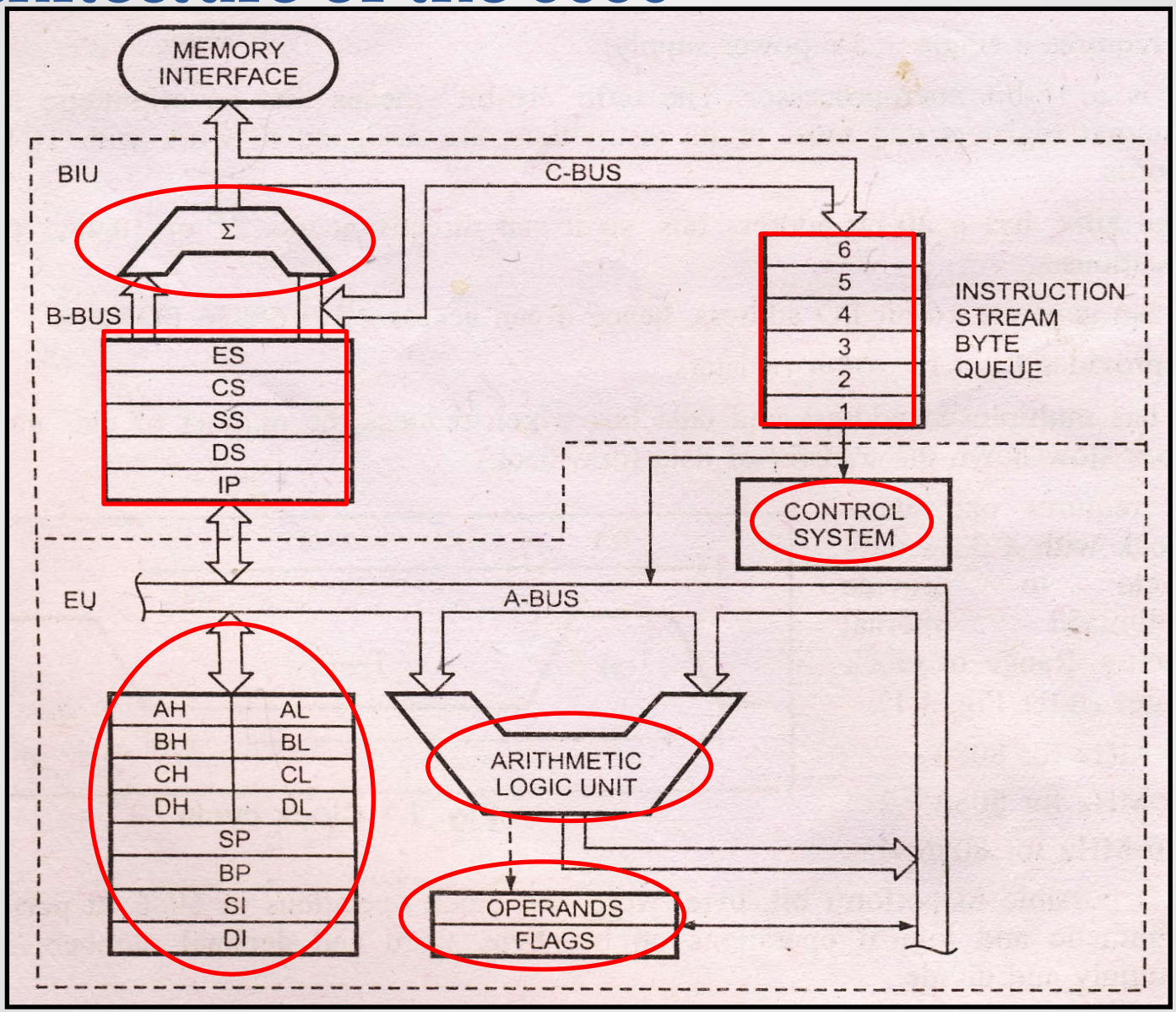
Cont'd…

- The 8086 can perform arithmetic and logical operations on bit, byte, word and blocks
- The 8086 is designed to operate in two modes
  - The minimum and (single processor microcomputers)
  - The maximum mode (multi processor appl)

- The 8086 supports multi-programming… time multiplexed
- it can fetch up to 6 instructions bytes from memory … to speed up instruction execution (pipelining)

- Provides powerful instruction sets with addressing modes like;
  - Register, Immediate, Direct, Indirect through index or base, Relative, and implied addressing mode

Architecture of the 8086



The Bus Interface Unit

- The BIU is the 8086's interface to the external world
- It provides a full bidirectional data bus and a 20-bit address bus
- The BIU is responsible for performing all external operations like;
  - Sends the address of the memory or IO
  - Fetches instruction from memory
  - Reads data from ports/memory
  - Writes data to ports/memory
  - Helps in instruction queuing
  - Provides address relocation

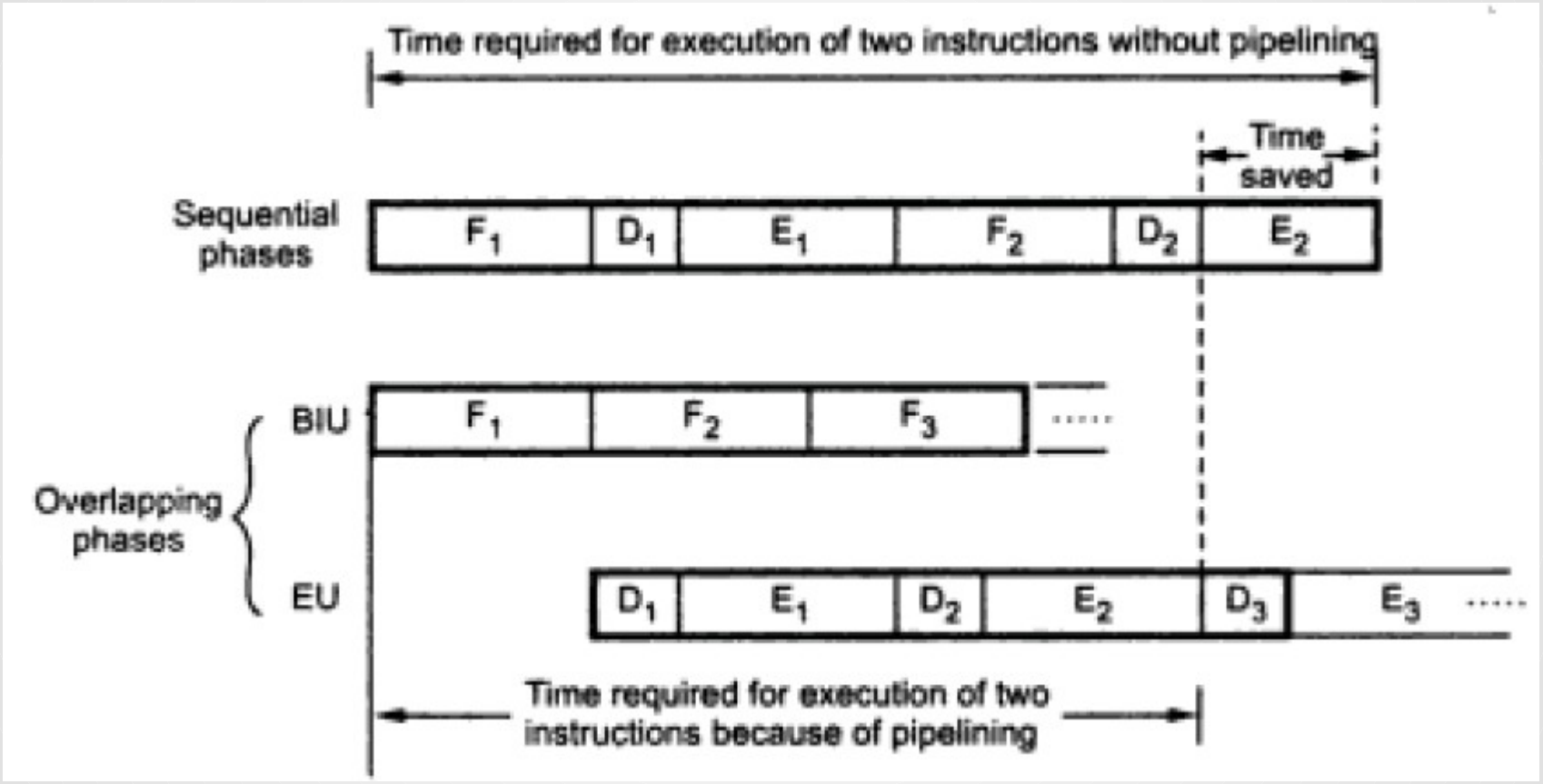
Instruction Queue

- To speed up program execution, the BIU fetches six instruction bytes ahead of time from memory
- These prefetched instruction bytes are held for the execution unit in a group of register called Queue

- it is possible to fetch next instruction while the current instruction is in execution … Pipelining
  - For example, current instruction in execution is a multiplication instruction

- Queue operates on the principle of FIFO
-

Pipelining


Time required for execution of two instructions without pipelining
Time saved
Sequential phases
F1
D1
E1
F2
D2
E2
BIU
F1
F2
F3
Overlapping phases
EU
D1
E1
D2
E2
D3
E3
Time required for execution of two instructions because of pipelining

Execution Unit (EU)

- The EU of 8086 tells the BIU where to fetch inst. or data from, decodes instructions & executes instruction

- **EU contains;**
  - control circuitry which directs the internal operations
  - decoder w/c translates the instructions fetched from memory into a series of actions
  - ALU w/c can add, subtract, AND, OR, XOR, increment, complement and shift binary numbers
  - Registers

Register Organization

- The 16-bit14 registers of the 8086 can be divided into:



General Purpose Registers

- 8086 has four 16-bit GPRs labeled AX, BX, CX & DX
- Each 16-bit GPRs can split into two 8-bit registers
- we use the letters L and H to specify the lower and the higher bytes of a particular register

- used for holding data, variables and intermediate results temporarily
- can also be used as counters or used for storing offset address for particular addressing modes
  - AX is used as a 16-bit accumulator
  - BX is used as offset storage for generating physical address
-

Segment Registers

- Physical address of 8086 is 20-bits wide to access 1Mbyte MLs
- However, its register that are meant to contain the address of ML are just 16-bit wide
- This brings about a question of addressing the 1Mbyte ML using only 16-bit
-
- Hence, 8086 uses memory segmentation that treats 1Mbyte ML as if they are divided into segments, with maximum segment size of 64Kbytes

- This implies that any location within the segment can be addressed using 16-bits of

Cont'd…

- The segment registers are used to hold the upper 16-bits of the starting address of the four segments, on which 8086 works on a particular time
- The starting address is to mean the lowest addressed byte in the active code segment

- The BIU always inserts nibbles (4-bits) in the contents of segment registers to generate 20-bit base address
  - e.g.

Functions of the segment registers

- The CS register holds the upper 16-bits of the starting address of the segment from which the BIU is currently fetching instruction code bytes

- The SS register is used for holding the upper 16-bits of the starting address for the program stack

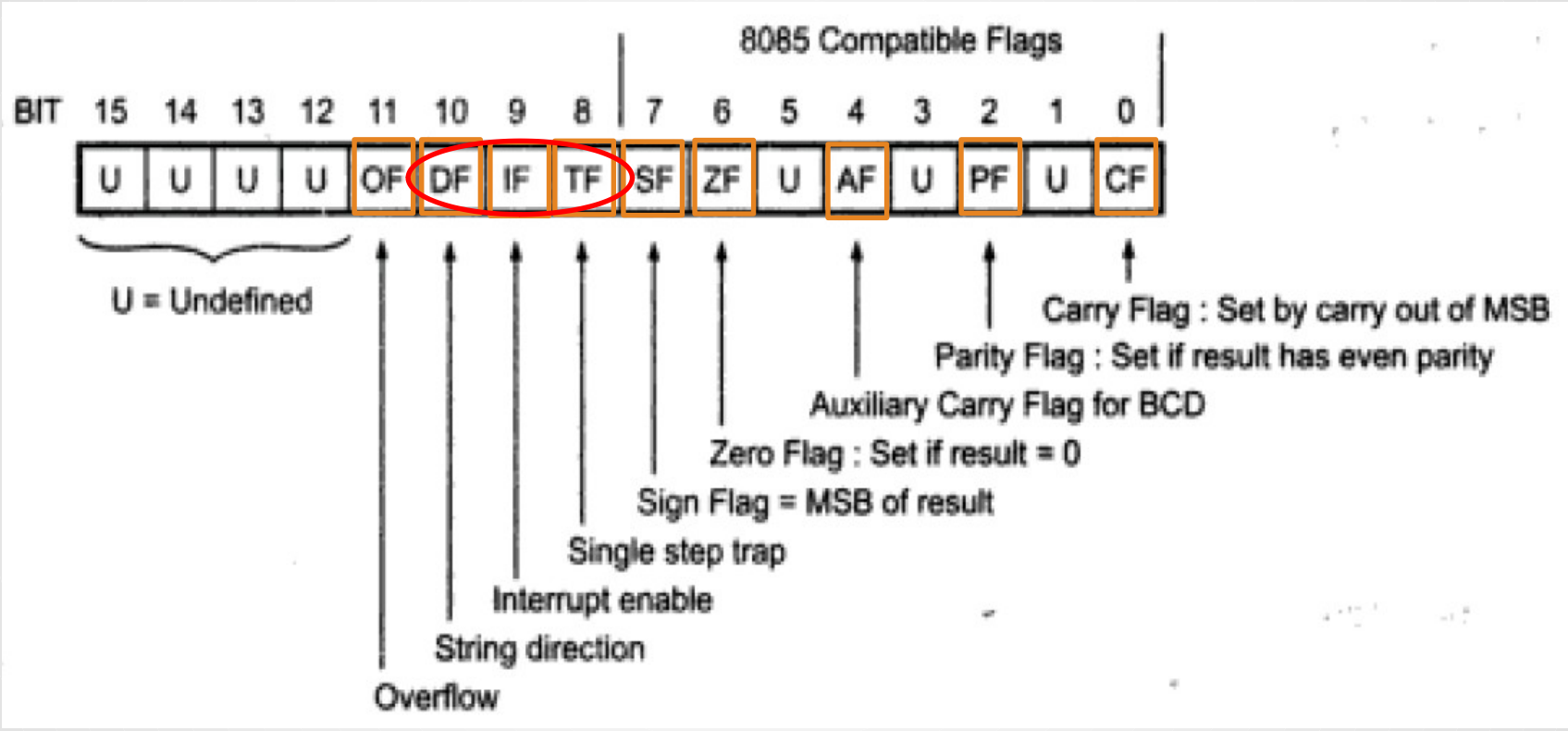
The ES & DS registers are used to hold the upper 16-bits of the starting address of the memory segments which are used for data.

Pointers and Index Registers

- All segment registers are 16-it wide
- it is necessary to generate 20-bit physical address on the address bus
- To get the 20-bit address, pointers or index registers are associated with each segment

- The pointer registers IP, BP, and SP are associated with code, data and stack segments respectively
- They all hold the offset value within the respective segments
- The index registers DI and SI are used as GPRs as well as for offset storage

Flag Register



1	**Carry Flag (CF)**	When addition instruction is executed, this bit (flag) will be set if carry is produced out of the MSB. This flag also serves as a borrow flag for subtraction and is set when borrow is needed.
2	**Parity Flag (PF)**	Is set to 1 if result of byte operation or lower byte of the word operation contains an even number of ones; otherwise it is zero.
3	**Auxiliary Flag (AF)**	This bit is set if there is carry from the lower nibble to the higher nibble (D3 bit to D4 bit). This flag is used for BCD operations and is not available for the programmer.
4	**Zero Flag (ZF)**	This flag is set of the result of operation in ALU is zero and resets if the result is non-zero. This flag also sets if the content of certain registers becomes zero following an increment or decrement operation of that register.
5	**Sign Flag (SF)**	After execution of arithmetic operations, if the MSB of the result is 1 the SF is set. This shows that the number is negative. Otherwise it is zero
6	**Overflow Flag (OF)**	Is set if result is out of range. For addition this flag is set when there is a carry into the MSB and no carry out of MSB or vice-versa. For subtraction, it is set when the MSB needs a borrow and there is no borrow from the MSB, or vice-versa.

Examples

#1. Give the content of the flag register after the execution of the following addition

```
  0110  0101  1101  0001
+ 0010  0011  0101  1001
  1000  1001  0010  1010
```

ANSWER:

SF=1, OF=1

ZF=0, AF=0

PF=1, CF=0

Cont'd…

#2. Give the contents of the flag register after execution of the following subtraction

```
  0110  0111  0010  1001
- 0011  0101  0100  1010
  ---------------------
  0011  0001  1101  1111
```

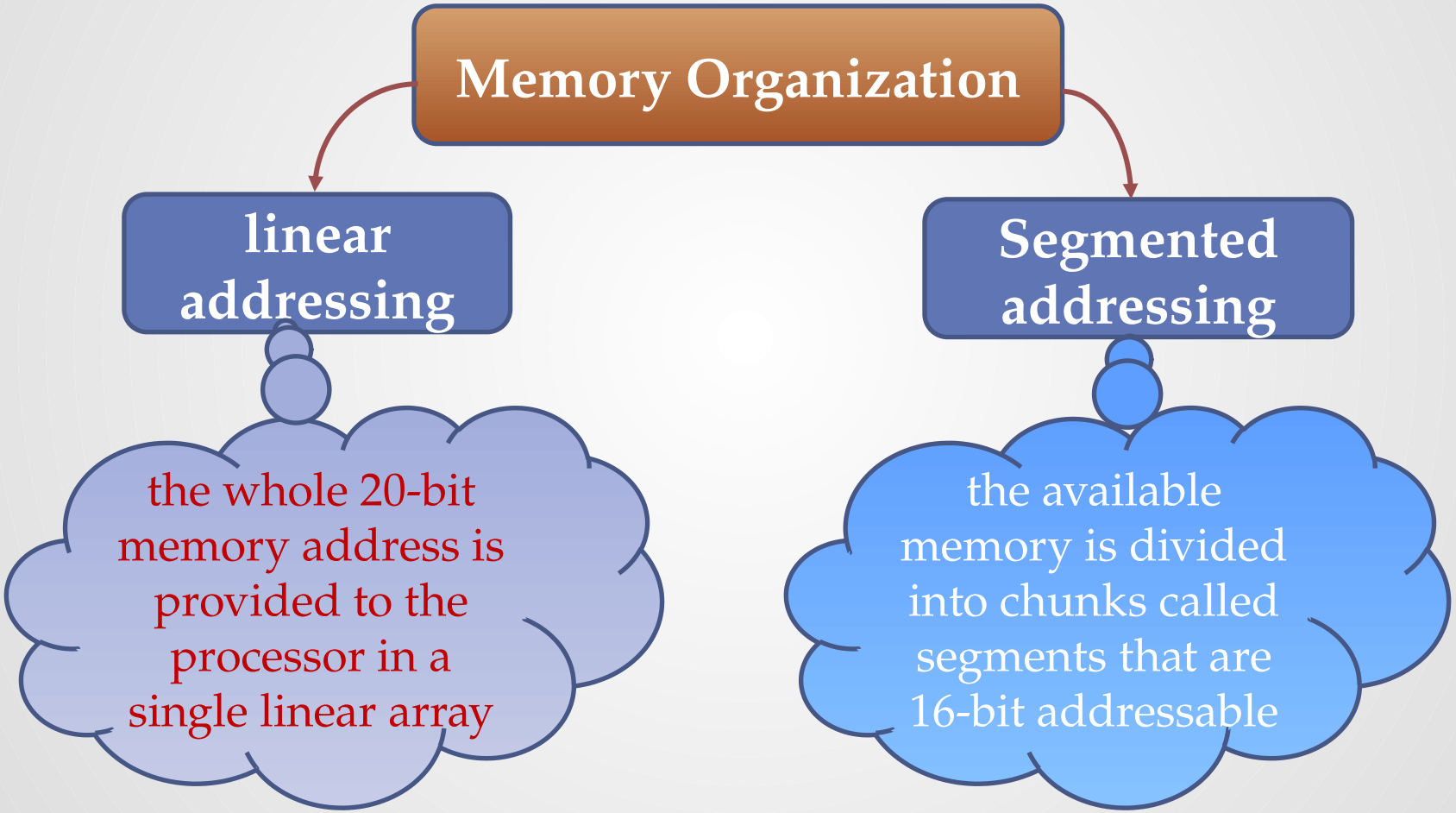
ANSWER:

SF=0, OF=0

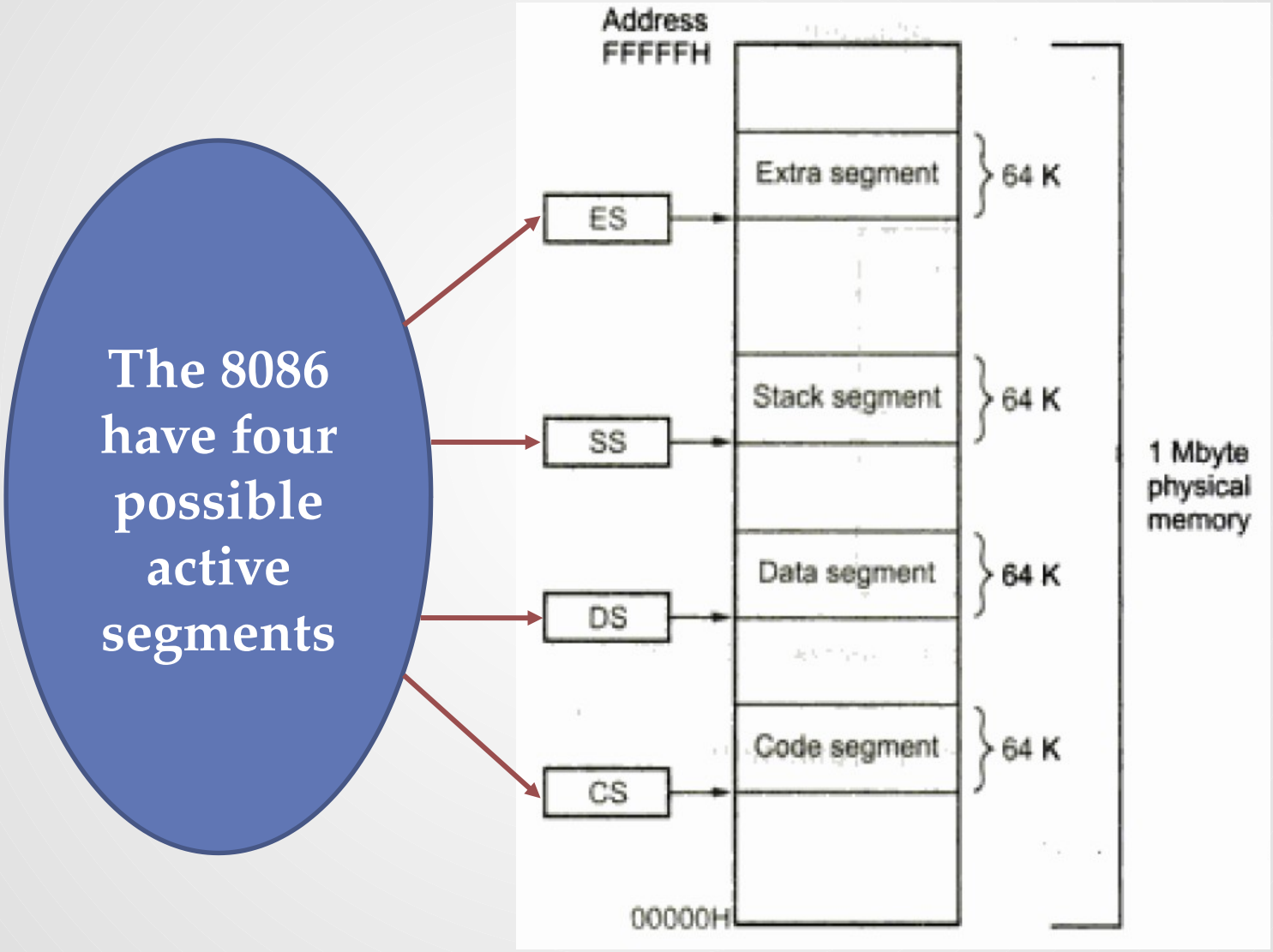
ZF=0, AF=1

PF=1, CF=0

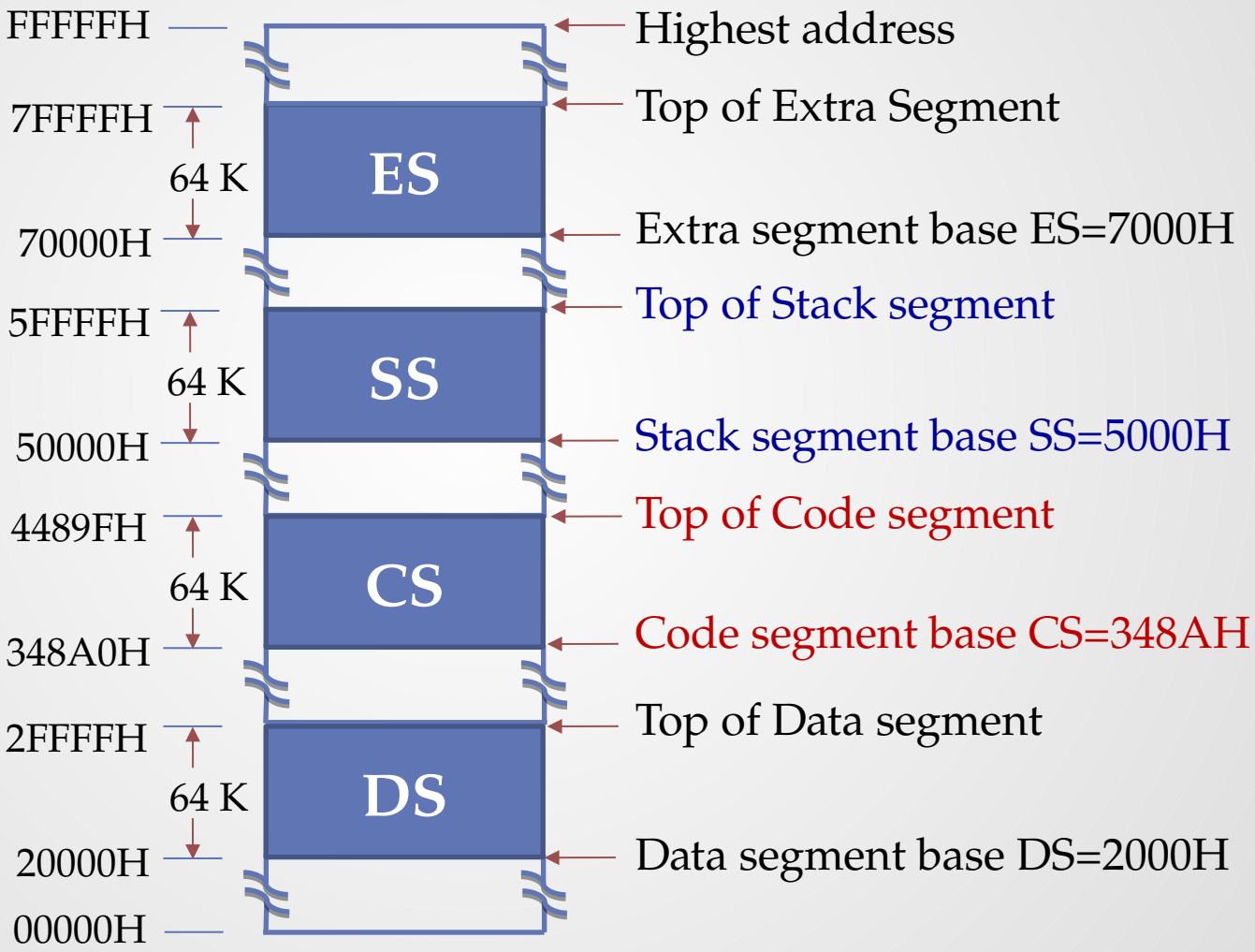
Memory Segmentation



Memory Segmentation & segment Registers



Memory Segmentation



Rules for Memory Segmentation

- The 4 segments can overlap for small programs
- The segment can start at any memory address w/c is divisible by 16

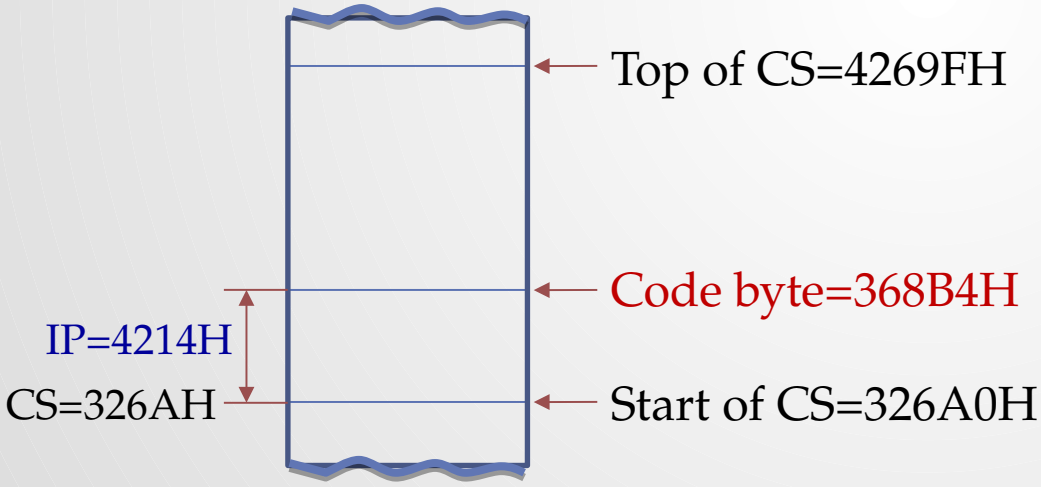
Advantages of MS

- Allows memory addressing capacity to be 1Mbyte
- Allows a portion of programs to be more than 64KB long by using more than one CS, DS, SS & ES
- Facilitates use of separate memory areas for program, data & stack
- Permits a program/its data to be put in d/t areas of memory, each time the program is executed

Example

3. Generation of 20-bit address using the code segment (CS) register and the Instruction pointer (IP)

Suppose the content of the code segment register is **326AH** and the IP is **4214H**



CS		3	2	6	A	0
IP	+		4	2	1	4
Physical Address		3	6	8	B	4

Default & Alternate Register Assignments

Type of Memory Reference	Default Segment	Alternate Segment	Offset (Logical Address)
Instruction fetch	CS	None	IP
Stack Segment	SS	None	SP, BP
General Data	DS	CS, ES, SS	Effective Address
String Source	DS	CS, ES, SS	SI
String Destination	ES	None	DI
BX used as pointer	DS	CS, ES, SS	Effective Address
BP used as pointer	SS	CS, ES, DS	Effective Address

Examples

Assume; CS=1000H, SS=3000H, BX=0020H,
ES=4000H, BP=0010H, DI=0050H,
DS=2000H, SP=0030H, SI=0040H,

1. MOV AL, [BP]

SS	30000H
BP	+ 0010H
Phy Add	**30010H**

This instruction copies a byte from memory location to the AL register. The effective address for the memory location is contained in the BP register. By default, an effective address is added to the SS to produce the physical memory address

Cont'd…

2. MOV AL, [BP+SI]

BP	0010H
SI	+ 0040H
Eff. Ad	**0050H**
SS	30000H
EA	+ 0050H
Phy Add	**30050H**

This instruction copies a byte from memory location to the AL register. The effective address is the summation of the contents of the BP & SI register.
The effective address is added to the SS to produce the physical memory address

Cont'd…

3. MOV CX, [BX] .....Exercise

4. MOV CS: [BX], AL

CS 10000H
BX + 0020H

Phy

This instruction copies a byte from AL register to a memory location. The effective address for the memory location is contained in the BX register. By default, an effective address in BX will be added to the DS to produce the physical memory address.

In this instruction, the CS: in front of [BX] indicates that we want BIU to add the effective address to the CS to produce the physical address. The CS: is called **segment override prefix**.

QUESTIONS?